KnowSphere AI — Enterprise Knowledge Intelligence Platform

Implementation Blueprint v1.0

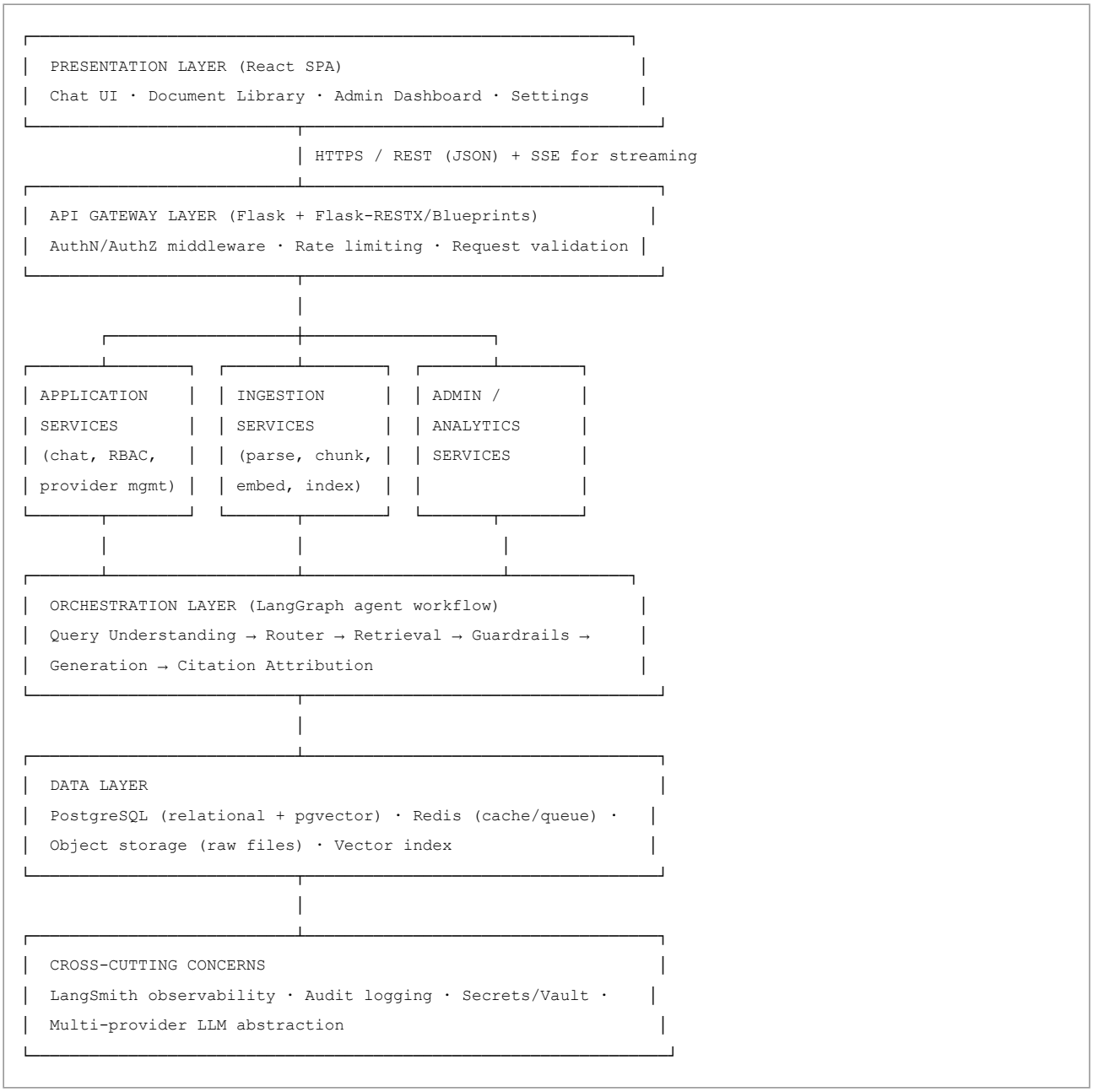
Prepared as a lead-architect-level design document. No code is included — this defines the shape of the system that subsequent build phases will implement against.

---

1. Overall Application Architecture

KnowSphere AI is built as a **layered, modular monolith with clean service boundaries** — structured so each layer can be extracted into an independent service later without a rewrite. This is the right starting point for a project at this stage: a microservices-first approach adds operational overhead (service discovery, distributed tracing, network latency) before there's a scaling reason to pay for it. The boundaries are drawn so that extraction is a refactor, not a redesign.

Layers, top to bottom:



Key architectural decisions:

- **Synchronous request path** handles chat queries end-to-end (retrieval + generation), streamed back via Server-Sent Events so the UI can render tokens as they arrive.
- **Asynchronous task queue** (Celery + Redis) handles document ingestion — parsing a 200-page PDF should never block an HTTP request thread. Upload returns immediately with a job ID; the UI polls or subscribes for status.
- **Stateless API layer** — all session state lives in Postgres/Redis, not in-process memory, so the API can scale horizontally behind a load balancer from day one.
- **Provider abstraction sits behind an interface**, not hardcoded to any one LLM vendor — this is what makes the "switch providers dynamically" requirement possible without touching business logic.

---

2. Folder Structure

```
knowsphere-ai/
├── backend/
│   ├── app/
│   │   ├── __init__.py                # App factory
│   │   ├── config.py                  # Env-based config classes
│   │   ├── extensions.py              # db, celery, cache init
│   │   │
│   │   ├── auth/
│   │   │   ├── routes.py              # login, SSO callback, token refresh
│   │   │   ├── models.py              # User, Session
│   │   │   ├── oidc.py                # OIDC/SAML integration
│   │   │   └── decorators.py          # @require_role, @require_permission
│   │   │
│   │   ├── rbac/
│   │   │   ├── models.py              # Role, Permission, ResourcePolicy
│   │   │   ├── service.py             # Permission resolution logic
│   │   │   └── routes.py              # Admin CRUD for roles/permissions
│   │   │
│   │   ├── documents/
│   │   │   ├── models.py              # Document, DocumentChunk, DocumentACL
│   │   │   ├── routes.py              # Upload, list, delete, reprocess
│   │   │   ├── parsers/
│   │   │   │   ├── pdf_parser.py
│   │   │   │   ├── docx_parser.py
│   │   │   │   ├── csv_excel_parser.py
│   │   │   │   ├── json_parser.py
│   │   │   │   ├── email_parser.py
│   │   │   │   └── chat_export_parser.py
│   │   │   ├── chunking.py            # Semantic chunking strategies
│   │   │   ├── metadata_extractor.py
│   │   │   └── tasks.py               # Celery tasks: parse→chunk→embed→store
│   │   │
│   │   ├── retrieval/
│   │   │   ├── embeddings.py          # Embedding provider abstraction
│   │   │   ├── vector_store.py        # pgvector/Qdrant client wrapper
│   │   │   ├── retriever.py           # Semantic + hybrid + filtered search
│   │   │   └── reranker.py            # Cross-encoder re-ranking (optional)
│   │   │
│   │   ├── agents/
│   │   │   ├── graph.py               # LangGraph state graph definition
│   │   │   ├── nodes/
│   │   │   │   ├── query_understanding.py
│   │   │   │   ├── router.py
│   │   │   │   ├── retrieval_node.py
│   │   │   │   ├── guardrails_node.py
│   │   │   │   ├── generation_node.py
│   │   │   │   └── citation_node.py
│   │   │   └── state.py               # Shared graph state schema
│   │   │
│   │   ├── chat/
│   │   │   ├── models.py              # ChatSession, ChatMessage, Citation
│   │   │   ├── routes.py              # Send message (SSE stream), history
│   │   │   └── service.py
│   │   │
```

```
│   │   ├── providers/
│   │   │   ├── base.py                # Abstract LLMProvider interface
│   │   │   ├── openai_provider.py
│   │   │   ├── anthropic_provider.py
│   │   │   ├── gemini_provider.py
│   │   │   ├── groq_provider.py
│   │   │   ├── openrouter_provider.py
│   │   │   ├── nvidia_nim_provider.py
│   │   │   ├── ollama_provider.py
│   │   │   ├── openai_compatible_provider.py
│   │   │   ├── registry.py            # Provider factory + validation
│   │   │   └── routes.py              # Settings page CRUD for provider configs
│   │   │
│   │   ├── security/
│   │   │   ├── secrets_manager.py     # Vault/KMS-backed key storage
│   │   │   ├── encryption.py          # Field-level encryption helpers
│   │   │   ├── pii_redaction.py
│   │   │   └── prompt_injection_guard.py
│   │   │
│   │   ├── audit/
│   │   │   ├── models.py              # AuditLog
│   │   │   ├── middleware.py          # Auto-log request/response metadata
│   │   │   └── routes.py              # Admin audit viewer
│   │   │
│   │   ├── analytics/
│   │   │   ├── routes.py              # Dashboard metrics endpoints
│   │   │   └── service.py             # Aggregation queries
│   │   │
│   │   ├── observability/
│   │   │   ├── langsmith_client.py    # Tracing wrapper around agent graph
│   │   │   └── tracing_middleware.py
│   │   │
│   │   └── common/
│   │       ├── errors.py              # Standardized error responses
│   │       ├── pagination.py
│   │       └── schemas.py             # Marshmallow/Pydantic shared schemas
│   │
│   ├── migrations/                    # Alembic migrations
│   ├── tests/
│   │   ├── unit/
│   │   ├── integration/
│   │   └── fixtures/
│   ├── celery_worker.py
│   ├── wsgi.py
│   ├── requirements.txt
│   └── Dockerfile
│
├── frontend/
│   ├── src/
│   │   ├── main.tsx
│   │   ├── App.tsx
│   │   ├── routes/                    # Route-level pages
│   │   │   ├── ChatPage.tsx
│   │   │   ├── DocumentsPage.tsx
```

```
│   │   │   ├── AdminPage.tsx
│   │   │   ├── SettingsPage.tsx
│   │   │   └── LoginPage.tsx
│   │   ├── components/
│   │   │   ├── chat/                  # MessageBubble, CitationChip, SourceDrawer
│   │   │   ├── documents/             # DocumentCard, UploadZone, RBACChips
│   │   │   ├── admin/                 # StatCards, AccessMatrix, AuditTable
│   │   │   └── shared/                # Nav, Modal, Toast, Button, Input
│   │   ├── hooks/
│   │   │   ├── useChatStream.ts       # SSE connection handling
│   │   │   ├── useAuth.ts
│   │   │   └── usePermissions.ts
│   │   ├── api/
│   │   │   ├── client.ts              # Axios/fetch instance with interceptors
│   │   │   ├── chat.ts
│   │   │   ├── documents.ts
│   │   │   ├── providers.ts
│   │   │   └── admin.ts
│   │   ├── store/                     # Zustand or Redux Toolkit slices
│   │   ├── types/                     # Shared TS types (mirrors backend schemas)
│   │   └── styles/
│   ├── public/
│   ├── package.json
│   └── vite.config.ts
│
├── infra/
│   ├── docker-compose.yml             # Local dev: postgres, redis, backend, frontend
│   ├── k8s/                           # Deployment manifests (future)
│   └── terraform/                     # IaC for cloud resources (future)
│
├── docs/
│   ├── architecture/                  # This document and diagrams
│   ├── api-spec/                      # OpenAPI/Swagger spec
│   └── runbooks/
│
└── .env.example
```

---

3. Technology Stack Justification

Layer	Choice	Why
Backend framework	**Flask** (+ Blueprints, Flask-RESTX)	Matches your stated stack; lightweight enough not to fight the agent/streaming requirements the way a more opinionated framework might.
Async/background jobs	**Celery + Redis**	Document ingestion is I/O- and CPU-heavy (parsing, embedding); must not block request threads. Redis doubles as cache and broker, reducing infra surface.
Frontend framework	**React + TypeScript + Vite**	Matches stated stack; TypeScript catches contract mismatches between frontend and backend schemas early — valuable given how many data shapes this app has (documents, citations, permissions).
Relational DB	**PostgreSQL**	Mature, supports JSONB for flexible metadata, and — critically — **pgvector** lets you keep embeddings and relational data (permissions, audit trails) in one transactional store rather than syncing two databases.

Layer	Choice	Why
Vector store	**pgvector (default) / Qdrant (optional swap-in)**	pgvector avoids running a second database for MVP and keeps ACL joins trivial (filter vectors by a SQL join on permissions). Qdrant becomes worth the operational cost only past tens of millions of vectors or when you need advanced filtering/performance pgvector can't give you — the abstraction in `retrieval/vector_store.py` makes this swappable.
Agent orchestration	**LangGraph**	Explicit state graph gives you controllable, debuggable multi-step reasoning (route → retrieve → guardrail → generate → cite) instead of an opaque agent loop — important for an enterprise tool where "why did it answer that way" must be answerable.
LLM abstraction	**LangChain provider integrations + custom `LLMProvider` interface**	LangChain gives working integrations for most providers out of the box; the custom interface on top keeps your app code decoupled from any one library's API surface, so a future LangChain breaking change doesn't ripple through the whole app.
Observability	**LangSmith**	Purpose-built for tracing LLM/agent workflows — captures prompts, retrieved chunks, tool calls, and latencies per run, which generic APM tools don't do well.
Auth	**OIDC/SAML via an identity provider (Okta/Azure AD/etc.)**, session via JWT + refresh token	Enterprises already have an IdP; the app should federate to it rather than own passwords — this is a security requirement, not just a convenience.
Secrets management	**HashiCorp Vault or cloud KMS (AWS Secrets Manager/Azure Key Vault)**	User-supplied LLM provider API keys are highly sensitive; they must never sit in plaintext in Postgres.
Containerization	**Docker + docker-compose (local), Kubernetes (production)**	Standard path for a system with multiple independently-scalable services (API, worker, frontend).
Document parsing	**unstructured.io, PyMuPDF/pypdf, python-docx, pandas, openpyxl**	`unstructured` gives a unified interface across many formats and handles messy real-world documents (tables, headers) better than format-specific libraries alone; format-specific libraries are kept as fallback/fine-grained control.

---

4. Backend Architecture

The backend is organized as **vertical slices** (auth, documents, retrieval, agents, chat, providers, audit, analytics) rather than horizontal layers (all models together, all routes together) — each folder in Section 2 is a self-contained domain with its own models, routes, and service logic. This keeps the codebase navigable as it grows, and means a new engineer working on "document ingestion" never needs to open the `chat/` folder.

Request lifecycle for a chat query:

1. `POST /api/v1/chat/sessions/{id}/messages` hits the API gateway.
2. Auth middleware validates the JWT, resolves the user and their role(s).
3. RBAC service computes the set of document IDs/collections this user may query — this is **not** left to the LLM to self-censor; it's a hard filter applied at the retrieval query level.
4. The request enters the LangGraph agent workflow (Section 8) with the permission filter injected into the graph state.
5. Retrieval node queries the vector store **with the permission filter as a SQL/metadata predicate**, so restricted chunks are never fetched, let alone shown to the model.
6. Generation node calls the configured LLM provider (Section 6, provider abstraction) with retrieved context + guardrail instructions.
7. Citation node maps generated claims back to source chunk IDs.
8. Response streams back over SSE; full message + citations persisted to Postgres; trace sent to LangSmith; audit log entry written.

Cross-cutting middleware (applied globally):

- Request ID generation (for tracing correlation across logs/LangSmith)
- Auth/session validation
- Rate limiting (per-user and per-org)
- Structured error handling → consistent JSON error envelope
- Audit logging hook (who did what, when, to which resource)

---

5. Frontend Architecture

Structure: route-based pages (Chat, Documents, Admin, Settings, Login) each composed of domain-specific components, following the same vertical-slice principle as the backend.

State management: a lightweight store (Zustand recommended over Redux Toolkit for this scope — less boilerplate, sufficient for the state shape here) holds:

- Current user/session/role
- Active chat session + message list (streamed in via SSE)
- Document library cache (with optimistic updates on upload/toggle)
- Provider/settings config

Key screens (mirroring the functional areas already validated in the prototype):

- **Chat** — streaming message list, inline citation chips opening a source drawer, session history sidebar, model/provider indicator.
- **Document Library** — upload (drag-drop, multi-format), per-document status (parsing/embedding/ready/failed), RBAC assignment UI, metadata view.
- **Admin Dashboard** — usage analytics, access control matrix, audit log viewer, ingestion job monitor.
- **Settings** — LLM provider management (add/validate/switch keys), LangSmith config, org-level guardrail toggles.

Design system: componentized (Button, Input, Modal, Toast, Table) with a shared token file (colors, spacing, type scale) so admin and chat surfaces feel like one product, not stitched-together screens.

Auth flow: SPA redirects to IdP for login; backend exchanges the auth code for tokens; frontend stores only a short-lived access token in memory (never localStorage, to reduce XSS exposure) and relies on an httpOnly refresh cookie.

---

6. Database Schema

Core tables (PostgreSQL, with `pgvector` extension enabled):

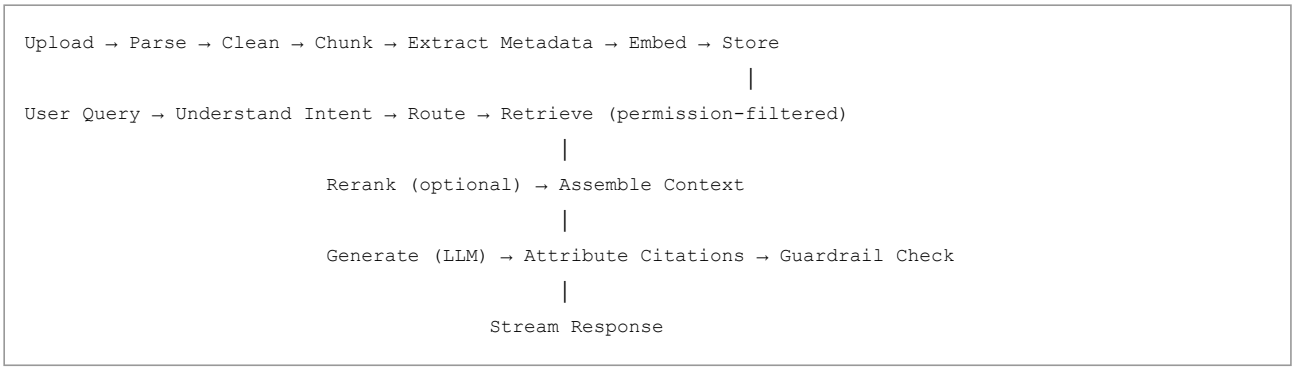
```
-- Identity & access
organizations(id, name, created_at)
users(id, org_id, email, display_name, idp_subject, created_at, last_login_at)
roles(id, org_id, name, description)                        -- e.g. Employee, Manager, HR Admin
permissions(id, code, description)                           -- e.g. document.read.confidential
role_permissions(role_id, permission_id)
user_roles(user_id, role_id)

-- Documents & knowledge
documents(id, org_id, title, source_type, uploaded_by, status,
          confidentiality_level, created_at, updated_at)
document_acl(document_id, role_id)                           -- which roles may access this doc
document_versions(id, document_id, version_no, storage_path, checksum, created_at)
document_chunks(id, document_id, chunk_index, content, token_count,
                embedding VECTOR(1536), metadata JSONB)
document_metadata(document_id, key, value)                   -- flexible extracted metadata

-- Chat
chat_sessions(id, user_id, title, created_at, updated_at)
chat_messages(id, session_id, role, content, provider_used, model_used,
              created_at)
citations(id, message_id, document_chunk_id, snippet, confidence_score)

-- Providers & settings
llm_provider_configs(id, org_id, provider_name, config_json,
                     secret_ref, is_active, validated_at)   -- secret_ref points to Vault, never the raw key
org_settings(org_id, key, value)

-- Observability & governance
audit_logs(id, org_id, user_id, action, resource_type, resource_id,
           metadata JSONB, ip_address, created_at)
ingestion_jobs(id, document_id, status, error_message,
               started_at, completed_at)
```

Design notes:

- `document_acl` is the enforcement point for RBAC at the data layer — retrieval queries always join through this table, so permission logic lives in one place, not scattered across application code.
- `llm_provider_configs.secret_ref` — the actual API key **never** lives in this table; only a reference/pointer into Vault/KMS does. This is non-negotiable for a system managing multiple third-party provider keys.
- `document_chunks.embedding` uses pgvector's `VECTOR` type with an IVFFlat or HNSW index for approximate nearest-neighbor search at scale.

---

7. Enterprise RAG Workflow

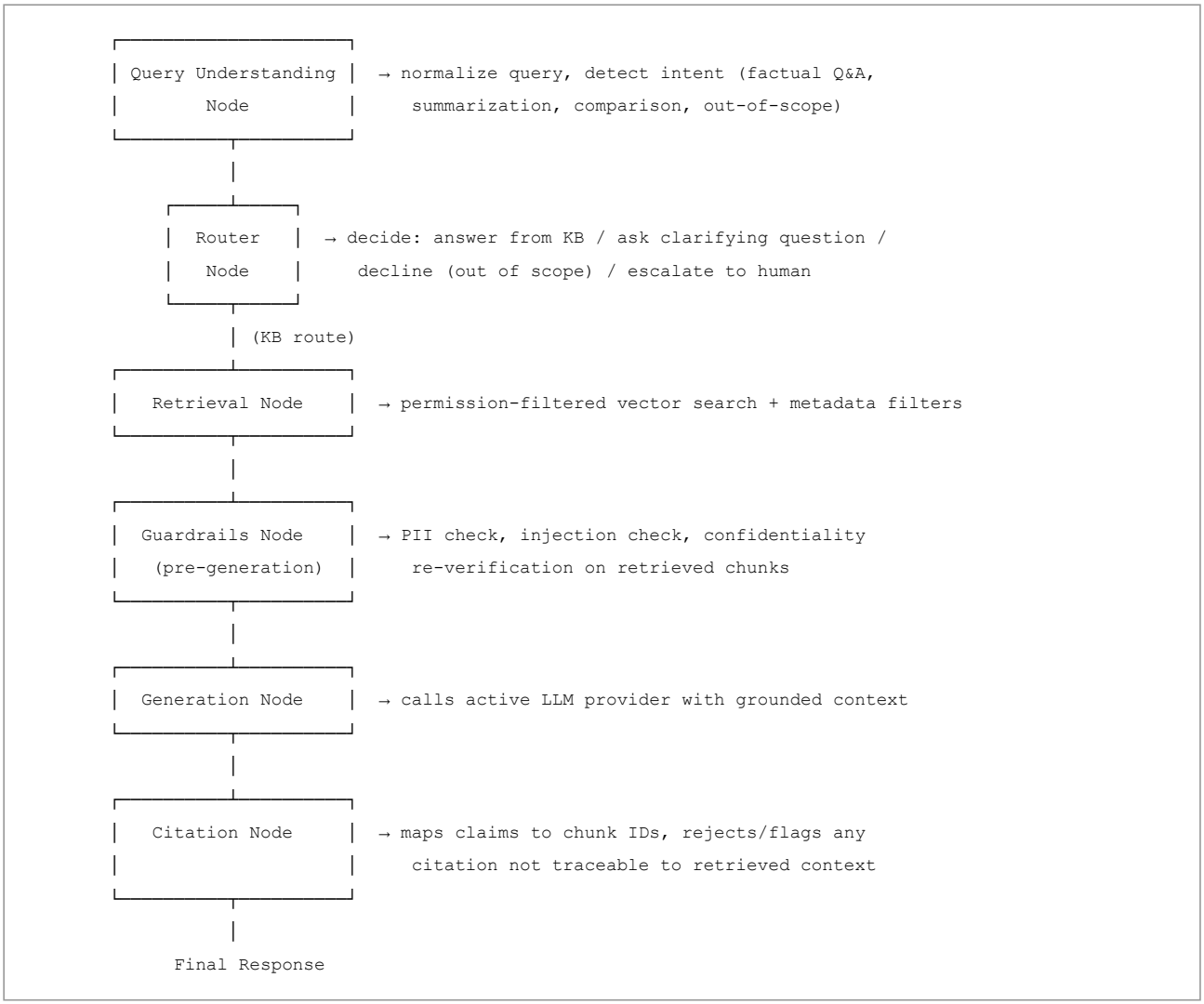


Why each stage exists:

- **Clean** — strip boilerplate (headers/footers, page numbers, OCR noise) before chunking, or embeddings get polluted with irrelevant repeated text.
- **Semantic chunking** (not fixed-size) — split on logical boundaries (headings, sections, paragraphs) so a chunk is a coherent unit of meaning; this directly improves citation precision, since a citation should point to one complete idea, not an arbitrary 500-token window.
- **Metadata extraction** — captures document type, department, confidentiality level, effective date, and section numbers (matching the § convention from the prototype) so retrieval can filter, and citations can reference a specific section.
- **Permission-filtered retrieval** — the single most important security control in the whole pipeline: the vector search query itself excludes chunks the user's role cannot see, so the LLM never receives forbidden content in its context window. Filtering after generation is not acceptable practice for this kind of system.
- **Rerank** (optional, phase 2+) — a cross-encoder re-scores the top-K retrieved chunks for relevance; worth adding once you observe retrieval quality issues with pure vector similarity, not before.
- **Guardrail check** — a lightweight pass (rule-based + a small classifier or LLM call) before the response is finalized, checking for prompt injection artifacts, policy violations, or hallucinated citations (a citation pointing to a chunk ID that wasn't actually retrieved).

---

8. LangGraph Agent Workflow



State object passed between nodes carries: original query, resolved user/role/permission set, retrieved chunks, intermediate reasoning notes, guardrail flags, and the provider/model selected — this is what LangSmith traces on, giving full visibility into *why* a given answer was produced.

Conditional edges matter here: the Router node can short-circuit to a "decline" or "clarify" response without ever hitting retrieval or generation — e.g., a query entirely unrelated to enterprise knowledge shouldn't burn a retrieval call or LLM token.

---

9. API Architecture

RESTful, versioned (`/api/v1/`), JSON request/response, SSE for streaming chat. Consistent error envelope: `{ "error": { "code", "message", "request_id" } }`.

Resource	Endpoints
Auth	`POST /auth/login` (SSO redirect), `GET /auth/callback`, `POST /auth/refresh`, `POST /auth/logout`
Users & Roles	`GET/POST /admin/users`, `GET/POST/PATCH /admin/roles`, `POST /admin/roles/{id}/permissions`
Documents	`POST /documents` (upload), `GET /documents`, `GET /documents/{id}`, `PATCH /documents/{id}` (toggle active, update ACL), `DELETE /documents/{id}`, `GET /documents/{id}/status`
Chat	`POST /chat/sessions`, `GET /chat/sessions`, `POST /chat/sessions/{id}/messages` (SSE stream), `GET /chat/sessions/{id}/messages`
Providers	`GET/POST /settings/providers`, `POST /settings/providers/{id}/validate`, `PATCH /settings/providers/{id}/activate`
Analytics/Admin	`GET /admin/analytics/usage`, `GET /admin/analytics/citations`, `GET /admin/audit-logs`

All endpoints enforce auth + RBAC at the route level via decorators (`@require_permission("document.write")`), in addition to the data-layer filtering described in Section 7 — defense in depth, not either/or.

10. Document Ingestion Pipeline

Format	Parser	Notes
PDF	`unstructured` (fallback: PyMuPDF)	Handles scanned PDFs via OCR fallback (Tesseract) when text layer is absent.
DOCX	`python-docx` / `unstructured`	Preserves heading structure for chunk boundaries.
TXT/MD	Native read + regex section detection	Matches the `§section` convention already validated in the prototype.
CSV/Excel	`pandas` + row/sheet-aware chunking	Row-level or table-level chunks with column headers retained as context.
JSON	Custom schema-aware parser	Flattens nested structures into semantically meaningful chunks; requires per-source schema mapping.
Email archives (.eml/.mbox/.pst)	`email` module / `libpff` for PST	Chunk per email thread; sender/date captured as metadata for filtering.
Slack/Teams exports	Custom JSON parser	Chunk per conversation thread, preserving participants as metadata (relevant for future ACL-by-channel).
SharePoint/Confluence	Connector APIs (Microsoft Graph, Confluence REST)	**Phase 2+**: live sync connectors; architecture supports this via a pluggable `Connector` interface mirroring the parser pattern.

Pipeline stages per document: upload → virus/malware scan → format detection → parse → clean → chunk (semantic, format-aware) → metadata extraction (confidentiality level, department, effective date) → embed (batch, via configured embedding provider) → store (chunks + vectors + metadata) → mark ready. Every stage writes to `ingestion_jobs` for status visibility in the UI, matching the "document status" experience from the prototype (parsing/embedding/ready/failed).

11. Security Architecture

- **AuthN**: OIDC/SAML federation to the org's identity provider. No locally-stored passwords.
- **AuthZ (RBAC)**: two-layer enforcement — (1) route-level permission checks, (2) data-layer ACL filtering on every retrieval query (Section 6/7). This is deliberate redundancy: a bug in one layer shouldn't mean a total bypass.
- **Confidentiality is never enforced by prompting alone.** "Don't mention document X" as a system-prompt instruction is a UX nicety, not a security control — the actual control is that restricted chunks are never retrieved into context in the first place.
- **Secrets management**: all third-party LLM/embedding API keys stored in Vault/KMS, referenced by pointer only; keys are never logged, including in LangSmith traces (redact before sending trace payloads).
- **Encryption**: TLS in transit everywhere; encryption at rest for the database and object storage; field-level encryption for anything classified confidential (e.g., compensation data) as defense in depth beyond disk encryption.
- **Guardrails**: prompt-injection detection on retrieved content (documents are untrusted input — a malicious or compromised document could contain injected instructions), PII redaction on outputs where appropriate, output validation that citations trace to actually-retrieved chunks (prevents hallucinated sources).
- **Audit logging**: every document access, permission change, and provider-config change is logged with actor, timestamp, and resource — required for enterprise compliance (SOC 2, ISO 27001) and for investigating any suspected confidentiality breach.
- **Multi-tenancy** (if serving multiple orgs): every table keyed by `org_id`, enforced at the query layer via a middleware that injects the tenant filter — never left to individual query authors to remember.

12. LangSmith Observability Plan

- **Trace every LangGraph run** end-to-end: query understanding → routing decision → retrieved chunk IDs → guardrail flags → generation → citation mapping — this is the full "why did it answer this way" audit trail.
- **Tag traces** with `user_role`, `org_id`, `session_id`, and `provider/model` so traces can be filtered by role (e.g., "show me all HR Admin queries this week") without exposing query content to the wrong audience.
- **Redact secrets and PII** from trace payloads before they leave the app — LangSmith should never receive a raw API key or unredacted employee PII in a trace.

- **Datasets & evaluation**: periodically export real (anonymized) query/response pairs into LangSmith datasets to build regression test suites — catches quality regressions when you swap providers or change prompts.
- **Cost & latency dashboards**: per-provider token usage and latency, since the multi-provider requirement means cost will vary significantly by which model answered a given query.
- **Alerting**: threshold alerts on error rate, guardrail-trigger rate, and latency p95 — feeding into whatever the org's existing on-call tooling is (PagerDuty, Opsgenie).

---

13. Development Phases

Phase 0 — Foundations (2–3 weeks) Auth (SSO), base RBAC model, Postgres schema, CI/CD skeleton, empty Flask/React shells wired together.

Phase 1 — Core RAG MVP (4–6 weeks) Single-format upload (PDF/TXT/DOCX), basic chunking + embedding + pgvector storage, simple retrieval + generation (single LLM provider — Anthropic, matching your existing prototype), chat UI with citations, document library with RBAC toggles. This phase should functionally match the validated prototype, now on real infrastructure.

Phase 2 — Multi-format & Multi-provider (4–5 weeks) CSV/Excel/JSON/email parsers, LangGraph agent workflow (router, guardrails, citation node) replacing the simple retrieval call, provider abstraction + settings UI for switching LLM providers, LangSmith integration.

Phase 3 — Enterprise Hardening (4–6 weeks) Full audit logging, Vault/KMS secrets integration, guardrails (prompt injection, PII redaction), analytics dashboard, reranking, ingestion job monitoring UI, load testing.

Phase 4 — Scale & Live Connectors (ongoing) SharePoint/Confluence/ERP live sync connectors, horizontal scaling (K8s), advanced multi-agent workflows (e.g., a summarization agent, a comparison agent), fine-grained document-version handling.

---

14. File-by-File Implementation Roadmap

This expands the Section 2 folder structure into build order — each item is a unit of work for a sprint-planning pass:

Phase 0: `auth/oidc.py`, `auth/models.py`, `rbac/models.py`, `rbac/service.py`, `common/errors.py`, `config.py`, `extensions.py`, frontend `LoginPage.tsx`, `hooks/useAuth.ts`.

Phase 1: `documents/parsers/pdf_parser.py`, `docx_parser.py`, `documents/chunking.py`, `retrieval/embeddings.py`, `retrieval/vector_store.py`, `retrieval/retriever.py`, `chat/models.py`, `chat/routes.py`, `providers/anthropic_provider.py`, `providers/base.py`, frontend `ChatPage.tsx`, `DocumentsPage.tsx`, `components/chat/*`, `components/documents/*`.

Phase 2: remaining parsers (`csv_excel_parser.py`, `json_parser.py`, `email_parser.py`, `chat_export_parser.py`), `agents/graph.py` and all `agents/nodes/*`, remaining `providers/*_provider.py`, `providers/registry.py`, `observability/langsmith_client.py`, frontend `SettingsPage.tsx`.

Phase 3: `security/secrets_manager.py`, `security/encryption.py`, `security/pii_redaction.py`, `security/prompt_injection_guard.py`, `audit/*`, `analytics/*`, frontend `AdminPage.tsx`, `components/admin/*`.

Phase 4: connector modules under a new `connectors/` package, `infra/k8s/*`, `infra/terraform/*`.

---

15. Best Practices and Potential Risks

Best practices:

- Enforce permission filtering at the data layer, never rely on prompt instructions alone (stated above, worth repeating — it's the single most common mistake in enterprise RAG builds).
- Treat every uploaded document as untrusted input for prompt-injection purposes, not just as a knowledge source.
- Keep the LLM provider abstraction thin and boring — resist the urge to special-case provider quirks deep in business logic; isolate quirks inside each provider adapter.

- Version documents rather than overwrite them, so citations remain valid even after a policy update (a citation to "PTO §2" should be traceable to the exact version that was active when the answer was given).
- Build the citation node to **reject** ungrounded claims (or flag them clearly) rather than let the model cite confidently to nothing — hallucinated citations are worse than no citation.
- Keep chunk size and overlap tunable per document type via config, not hardcoded — a Slack export and a 40-page policy PDF want different chunking.

Risks and mitigations:

Risk	Mitigation
Hallucinated or ungrounded citations	Citation node validates every cited chunk ID against the actual retrieved set before returning the response.
Prompt injection via malicious/compromised documents	Guardrails node scans retrieved chunks for injection patterns before they reach the generation prompt.
Confidential data leakage across roles	Enforced at the vector-query layer (data-level ACL), not just the system prompt; tested via automated cross-role access tests in CI.
LLM provider API key exposure	Vault/KMS-backed storage, redaction in logs/traces, least-privilege access to secrets.
Vendor lock-in to a single LLM provider	Provider abstraction interface designed from day one (Section 3/6), even though Phase 1 ships with a single provider.
Cost overrun from token usage at scale	Per-provider cost tracking in LangSmith dashboards, caching of repeated queries, tunable retrieval top-K.
Retrieval quality degradation as document volume grows	Reranking stage planned for Phase 2+; monitor retrieval precision via the LangSmith eval dataset.
Compliance exposure (GDPR/SOC 2) from audit gaps	Comprehensive audit logging from Phase 0, not bolted on later; regular access reviews of `document_acl`.
Ingestion pipeline failures on malformed real-world documents	`ingestion_jobs` status tracking with clear failure reasons surfaced in the UI, so failures are visible rather than silent.

This blueprint is the reference for all subsequent build phases. Recommend treating Sections 6 (schema) and 11 (security) as the least negotiable — changes there tend to cascade expensively once implementation begins.